

Abstract

Guitar tablature is the most widely used notation system among guitarists, yet generating it automatically from natural language descriptions remains an open problem. General purpose large language models tend to hallucinate token formats when asked to produce structured music notation, motivating the use of fine-tuning for this domain. In this work, we collect 262 GuitarPro files, build a custom tokenization pipeline using PyGuitarPro, and fine-tune TinyLlama 1.1B and Mistral 7B using LoRA on paired natural language descriptions and token sequences. We introduce two evaluation metrics: structural validity rate, which measures token level format compliance, and technique adherence rate, which measures whether requested guitar techniques appear in the generated output. We evaluate against GPT-4o as a baseline and find that our fine-tuned Mistral 7B model achieves 97.46% structural validity compared to 88.03% for GPT-4o, while GPT-4o outperforms our model on technique adherence (93.75% vs 31.25%).

Introduction

Guitar tablature is the most popular form of guitar music transcription. It is a simple and intuitive system that encodes the physical mechanics of playing the instrument, showing the guitarist which fret to play on which string, which techniques like slides, bends, or hammer-ons are used, along with things like pitch and rhythm. This makes tablature a uniquely structured output format with its own grammar and constraints that are different from standard music notation. GuitarPro is the most widely used software for creating and editing guitar tablature. Its file format stores all of this information in a structured binary format that can be parsed programmatically, making it a natural source of data for a generative model.

The goal of this project is to fine-tune a language model that accepts natural language descriptions as input and returns structurally valid, technically playable guitar tablature that follows the instructions of the prompt. For example, given a prompt like "give me a blues riff at 80 BPM with bends and vibrato," the model should produce a token sequence that encodes a coherent guitar part at the right tempo using the requested techniques.

To represent tablature in a format suitable for language model training, we define a custom token vocabulary inspired by the DadaGP dataset. Each token encodes a discrete musical event: for example, *note:s4:f7:eighth* represents an eighth note played on string 4 at fret 7, *hammer* indicates a hammer-on technique applied to the preceding note, and *wait:quarter* represents a quarter note rest. A complete guitar part is encoded as a flat sequence of these tokens, similar to how source code can be represented as a sequence of text tokens for a code generation model.

General purpose large language models like GPT-4o are capable of tablature generation, but they tend to hallucinate formatting and invent their own token variations rather than adhering to a consistent schema.

This is a well known failure mode for LLMs when asked to generate constrained structured outputs: the model understands the concept but does not respect the exact format required.

Related Work

The most relevant work to this project is DadaGP (Sarmiento et al., 2021), which introduced a dataset of 26,181 GuitarPro songs converted into a flat token sequence format designed for sequence models. The key contribution is the tokenization scheme itself, which encodes notes, timing, and techniques as discrete text tokens. Our work is directly inspired by this approach but fine-tunes a pretrained LLM rather than training from scratch, and adds natural language conditioning.

ShredGP (Sarmiento et al., 2023) extended DadaGP with guitarist style conditioning, demonstrating that conditioning on musical attributes during generation is feasible within the GuitarPro token format. Our work continues this idea by replacing discrete style labels with open ended natural language prompts.

ChatMusician (Yuan et al., 2024) is the most methodologically similar work, fine-tuning LLaMA on ABC music notation and demonstrating that LLMs can learn music notation as a language when trained on domain specific data. Our work extends this paradigm from ABC notation to GuitarPro token format, adding guitar specific complexity around string and fret encoding and technique tokens.

MusicGen (Copet et al., 2023) takes a contrasting approach, generating audio waveforms directly from natural language prompts rather than symbolic notation. While our work shares the goal of natural language conditioned music generation, our symbolic output is more directly useful for guitarists who want to read and perform the generated music.

Data and Tokenization

Our dataset consists of 262 GuitarPro files collected from Songsterr, spanning a range of genres including rock, metal, blues, jazz, punk, and country. We specifically selected tabs that only had guitar rather than full band scores to keep the token space focused on guitar notation. Six files were excluded during preprocessing due to encoding errors in PyGuitarPro, leaving 256 files for processing. We used PyGuitarPro 0.6 to parse each GuitarPro file into a structured Python object containing tracks, measures, beats, and notes. From each note we extract the string number (1-6, where 6 is the low E string and 1 is the high e string), the fret number (0-24), the duration, and any technique effects applied to the note. We then convert this structured representation into a flat sequence of text tokens using our custom tokenizer. The full token vocabulary consists of the following types:

- *tempo:{BPM}* — song tempo, appears once at the start
- *new_track* — marks the beginning of a new instrument track
- *new_measure* — marks the beginning of a new measure
- *chord_start / chord_end* — wraps notes played simultaneously
- *note:s{1-6};f{0-24};{duration}* — encodes a single note with string, fret, and duration
- *wait:{duration}* — encodes a rest or time advance

- Technique tokens: *hammer, bend, slide, vibrato, let_ring, palm_mute, staccato, tremolo_pick, trill, harmonic*

Valid duration values are: whole, half, quarter, eighth, sixteenth, thirty_second. An example token sequence for a simple two note phrase with a hammer-on looks like: *tempo:120 new_track new_measure note:s4:f5:eighth hammer wait:eighth note:s3:f7:quarter wait:quarter*. For each GuitarPro file we programmatically generate a natural language description from the metadata extracted during parsing. The description includes the song tempo, instrument track names, detected techniques, and song length in measures. For example: *Tempo: 141 BPM. Instruments: Rhythm Guitar, Lead Guitar. Techniques: bend, hammer-on, slide, palm mute. Length: 125 measures*. These descriptions serve as the input side of our training pairs. Each training example is formatted as an instruction following prompt with the token schema included in the system instruction, the natural language description as the input, and the token sequence as the expected response.

Building a clean dataset required several iterations. We discovered that many songs begin with several empty measures before the guitar part enters, resulting in training outputs that started with long sequences of *wait* tokens. The model learned to associate descriptions that had a lot of different techniques with these openings with *wait* occurred, which contributed to poor technique adherence at inference time. We addressed this by stripping leading wait tokens from each training example, starting the output at the first note token instead. Finally, we applied a technique matching filter that removes any training example where a technique mentioned in the natural language description does not appear anywhere in the token sequence output. This ensures the model is only trained on examples where the input and output are consistent with each other.

A notable characteristic of our dataset is the highly imbalanced distribution of technique tokens relative to note tokens. Table 1 shows the frequency of each technique as a percentage of total notes across the training set.

Technique	Count	% of Notes
let_ring	73,352	5.12%
palm_mute	71,245	4.97%
staccato	27,212	1.90%
slide	23,932	1.67%
hammer	16,492	1.15%
vibrato	11,350	0.79%
bend	9,137	0.64%

harmonic	1,523	0.11%
tremolo_pick	911	0.06%
trill	0	0.00%

Table 1

Even the most common techniques appear on fewer than 6% of notes, meaning the overwhelming majority of note tokens are not followed by any technique modifier. This class imbalance has significant implications for technique adherence at inference time, which we discuss in the Results section.

Methods

We experimented with two base models. We initially used TinyLlama 1.1B as a lightweight starting point to validate our training pipeline before committing to a larger model. After observing consistent overfitting and poor technique adherence with TinyLlama, we switched to Mistral 7B v0.1 as our primary model. Mistral 7B has more capacity to learn the mapping between natural language descriptions and token sequences. We fine-tune both models using Low-Rank Adaptation (LoRA) which allows us to adapt the model to our domain without updating all 7 billion parameters. We apply LoRA to the query, key, value, and output projection layers with rank $r=16$ and scaling factor $\alpha=32$, resulting in approximately 4.5 million trainable parameters out of 1.1 billion for TinyLlama and a similar proportion for Mistral 7B. We use the HuggingFace PEFT and TRL libraries for training, with the SFTTrainer for supervised fine-tuning.

Each training example is formatted as a prompt with three components: a system instruction containing the full token schema, the natural language description as the input, and the token sequence as the expected response. Including the token schema in every training example was an important design decision since early experiments without the schema in the prompt resulted in significantly worse structural validity. For Mistral 7B we train for 3 epochs with a batch size of 1, gradient accumulation steps of 8, and a learning rate of $2e-4$. For TinyLlama we experimented with 3, 5, and 15 epochs, finding that 15 epochs caused severe overfitting where the model produced identical outputs regardless of the input prompt.

We use GPT-4o as our baseline, prompted with the same natural language descriptions as our fine-tuned model. Importantly, we include the full token schema in the GPT-4o prompt at inference time, giving it explicit knowledge of the valid token vocabulary. This represents the strongest possible general-purpose LLM baseline since GPT-4o has access to the same format specification our model was trained on. Any advantage our fine-tuned model shows over this baseline can therefore be attributed to learning the token grammar from examples rather than from an explicit specification.

We evaluate both models on two metrics we designed specifically for this task. Structural validity rate measures what percentage of generated tokens conform to our defined vocabulary, and technique

adherence rate measures what percentage of techniques explicitly requested in the natural language prompt appear in the generated output.

Results

Table 2 shows structural validity scores for our fine-tuned Mistral 7B model and GPT-4o across eight manually designed prompts covering a range of genres and techniques.

Model	Average Structural Validity	Minimum	Maximum
Fine-tuned Mistral 7B	97.46%	96.15%	100%
GPT4-o	88.03%	51.28%	100%

Table 2

Our fine-tuned model achieves an average structural validity of 97.46% compared to 88.03% for GPT-4o, but more notable than the average is the consistency: our model never falls below 96.15% across any prompt while GPT-4o drops as low as 51.28% on some inputs. GPT-4o's lower scores stem from inconsistent format adherence even when the schema is explicitly provided, with common failure modes including combining technique modifiers directly into the note token (for example *note:s6:f5:eighth:hammer* instead of a separate hammer token), using commas as separators instead of spaces, and wrapping output in markdown code blocks.

Table 3 shows technical adherence to our prompt for both models across the same eight prompts.

Model	Average Technique Adherence	Minimum	Maximum
Fine-tuned Mistral 7B	31.25%	0.00%	100%
GPT4-o	93.75%	50.0%	100%

Table 3

GPT-4o significantly outperforms our fine-tuned model on technique adherence, achieving 93.75% on average compared to 31.25%. GPT-4o never completely fails on any prompt, with a minimum of 50%, while our model scores 0% on several prompts. This result reflects a fundamental limitation of our fine-tuning approach: the model learned the token grammar well but did not reliably learn to connect natural language technique requests to the corresponding output tokens.

Further analysis revealed that technique adherence does not correlate with token frequency in the training data. Bend, which appears on only 0.64% of notes, and let ring, which appears on 5.12% of notes, are both reliably generated. Meanwhile palm mute (4.97%) and hammer-on (1.15%) are not. This suggests that contextual placement and co-occurrence patterns with specific note types are more influential than raw frequency.

We also investigated the effect of inference temperature on the tradeoff between structural validity and technique adherence. Higher temperatures improve technique adherence by introducing more randomness into the generation process, but begin to degrade structural validity above 1.1. At temperature 1.3 the model starts generating invalid tokens like *waiting:sixteenth*, a hallucinated variation of the valid *wait:sixteenth* token. Temperature 1.1 represents the best tradeoff for this prompt, maintaining 97% structural validity while improving technique adherence.

Discussion, Limitations, and Conclusion

Our results reveal a clear tradeoff between format compliance and musical expressiveness. Fine-tuning on domain specific token data significantly improves structural validity, but does not reliably teach the model to respond to technique requests in natural language prompts. GPT-4o demonstrates the opposite profile: strong technique adherence due to its capabilities of following instructions, but inconsistent format compliance due to never having been trained on our token schema.

The most significant limitation of our approach is the class imbalance in technique tokens. Even the most common techniques appear on fewer than 6% of notes, meaning the model is overwhelmingly trained to generate note wait pairs without technique modifiers. This is compounded by the fact that technique adherence does not correlate with frequency, rather contextual placement and co-occurrence patterns appear to play a larger role.

A second limitation is model size. TinyLlama 1.1B showed consistent overfitting even at 5 epochs, producing identical outputs across diverse prompts. Mistral 7B generalized better but still showed signs of memorization. Larger models or more training data would likely improve prompt responsiveness. Finally, our evaluation metrics measure format compliance and technique presence but not musical quality or playability. Future work should include human evaluation, oversampling of rare technique tokens, and larger base models.

References

- Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., & Défossez, A. (2023). Simple and Controllable Music Generation. *Advances in Neural Information Processing Systems*.
- Sarmiento, P., Kumar, A., Chen, Y.-H., Carr, C.J., Zukowski, Z., & Barthet, M. (2021). DadaGP: A Dataset of Tokenized GuitarPro Songs for Sequence Models. *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*.
- Sarmiento, P., Kumar, A., Xie, D., Carr, C.J., Zukowski, Z., & Barthet, M. (2023). ShredGP: Guitarist Style-Conditioned Tablature Generation. *arXiv:2307.05324*.
- Yuan, R., Lin, H., Wang, Y., Tian, Z., Wu, S., Shen, T., Zhang, G., Wu, Y., Liu, C., & Zhou, Z. (2024). ChatMusician: Understanding and Generating Music Intrinsically with LLM. *Findings of ACL 2024*.